

AI FitTrack Final Project Report

Phase 7

This final report presents the complete backend project from problem definition through architecture, implementation, API design, security, Gemini integration, testing, outcomes and submission requirements.

Field	Value
Project	AI FitTrack
Submission	SmartBridge Naan Mudhalvan
Project scope	Backend-only REST API
Technology	Node.js, Express.js, MongoDB, Mongoose, JWT and Google Gemini AI
Source baseline	Latest uploaded backend ZIP and supplied verification evidence
Document status	Submission-ready content; student metadata and final video links remain pending

Prepared for academic evaluation. Secrets, credentials, private environment files and unverified claims are excluded.

Document contents

1. Abstract and project overview
2. Problem statement and objectives
3. Requirements and scope
4. System design
5. Database and API specification
6. Implementation
7. Security and AI controls
8. Testing and results
9. Limitations and future scope
10. Conclusion
11. Appendices

Abstract

AI FitTrack is a secure backend REST API for personal workout tracking and AI-assisted fitness guidance. It is implemented with Node.js, Express.js, MongoDB, Mongoose, JSON Web Tokens, bcrypt and Google Gemini AI. Authenticated users can maintain an owner-isolated workout history, search that history and request structured recommendations or progress insights. The supplied evidence records successful live authentication, CRUD, search and Gemini flows together with 95 automated passes, 17 smoke checks and 19 Atlas checks. The project is backend-only and is ready for integration with a future web or mobile client.

1 Introduction

1.1 Problem statement

Manual workout notes are difficult to search, update and protect. They also provide little support for turning basic exercise history into practical guidance. AI FitTrack addresses those needs through a centralized, authenticated API rather than a user-interface-specific implementation.

AI FitTrack problem to solution map

Backend-only scope derived from the supplied requirements and source code

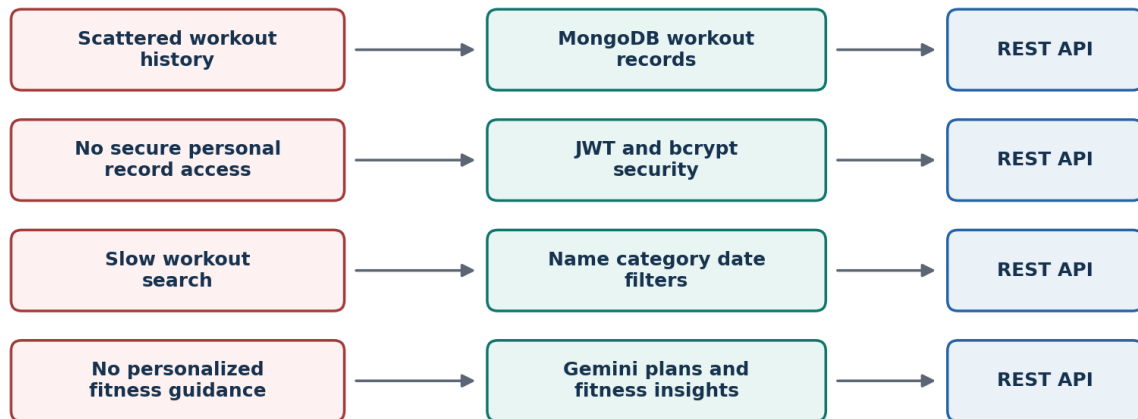


Figure 1 Problem to solution mapping

1.2 Objectives

- Protect personal fitness data through secure authentication and owner-scoped queries.
- Provide complete workout creation, retrieval, update, deletion and search functions.
- Use validated MongoDB documents for reliable workout history.
- Use Gemini to create structured recommendations and interpret workout statistics.
- Expose stable JSON contracts for later frontend or mobile integration.
- Handle invalid input, unknown routes, database failures and AI failures predictably.

1.3 Scope

Included scope comprises server configuration, database access, authentication, profile retrieval, workout CRUD and search, Gemini recommendation and insights, centralized validation and errors, tests and Postman assets. A React frontend, nutrition features, wearables, reminders and production deployment are outside this backend-only submission.

2 Requirement Analysis

Functional requirement groups

All protected features are scoped to the authenticated user

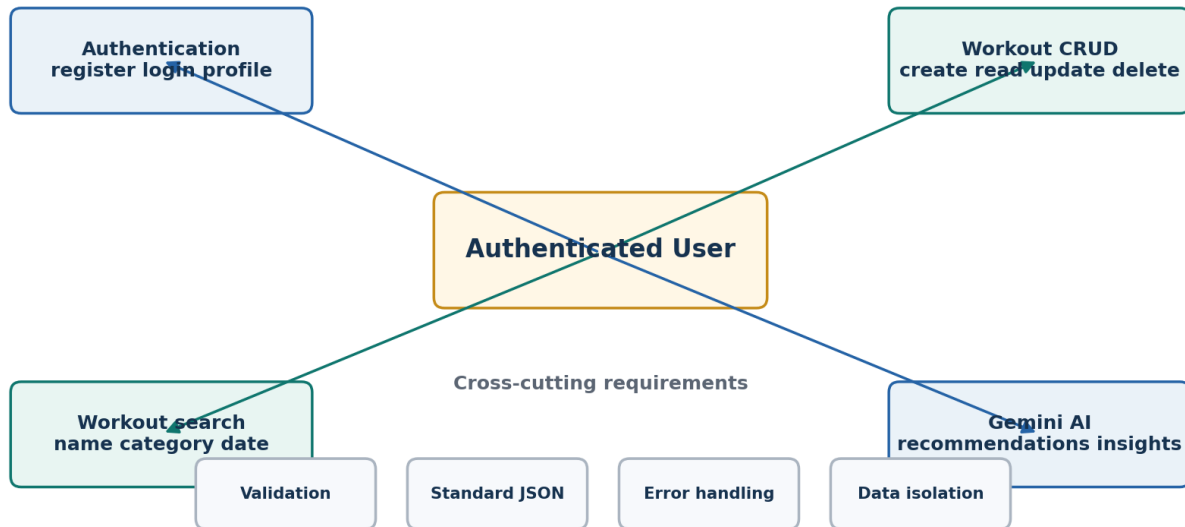


Figure 2 Functional requirement groups

ID	Requirement	Verified status
R1	Layered MVC-style backend with routes, controllers, models, middleware, services and config	Implemented
R2	Node.js and Express.js REST API	Implemented
R3	MongoDB and Mongoose connection management	Implemented
R4	Users and workouts collections	Implemented
R5	User schema with validation, unique email, bcrypt and timestamps	Implemented
R6	Workout schema with owner, fields, validation and timestamps	Implemented
R7	User registration and duplicate handling	Implemented and live-evidenced
R8	User login and JWT issuance	Implemented and live-evidenced
R9	Protected user profile	Implemented and live-evidenced

ID	Requirement	Verified status
R10	Create owner-scoped workout	Implemented and live-evidenced
R11	List workout history	Implemented and live-evidenced
R12	Search by name, category and date	Implemented and live-evidenced
R13	Get workout by identifier	Implemented and live-evidenced
R14	Partial workout update	Implemented and live-evidenced
R15	Delete workout and deny later retrieval	Implemented and live-evidenced
R16	Gemini workout recommendation	Implemented and live-evidenced
R17	Gemini fitness insights using submitted or database statistics	Implemented and live-evidenced
R18	Standard JSON responses and centralized error handling	Implemented
R19	Request and schema validation	Implemented and automated-test evidenced
R20	JWT middleware and fresh user lookup	Implemented and automated-test evidenced
R21	Password protection and response redaction	Implemented and live-evidenced
R22	Per-user data isolation	Implemented and automated-test evidenced
R23	Postman, automated, smoke and Atlas verification	Evidence supplied
R24	Environment-based MongoDB, JWT and Gemini configuration	Implemented with example only

2.1 Nonfunctional requirements

- Security through hashing, JWT verification, owner isolation, safe serialization and private environment values.
- Maintainability through modular routes, controllers, models, middleware, services, configuration and utilities.
- Reliability through database-gated startup, centralized errors, AI timeouts and bounded retries.

- Interoperability through HTTP, JSON, CORS and documented API contracts.
- Testability through validation units, Supertest integration, memory MongoDB, smoke scripts, Atlas scripts and Postman.

3 System Design

Implemented backend architecture

Request path and service boundaries in the uploaded Node.js project

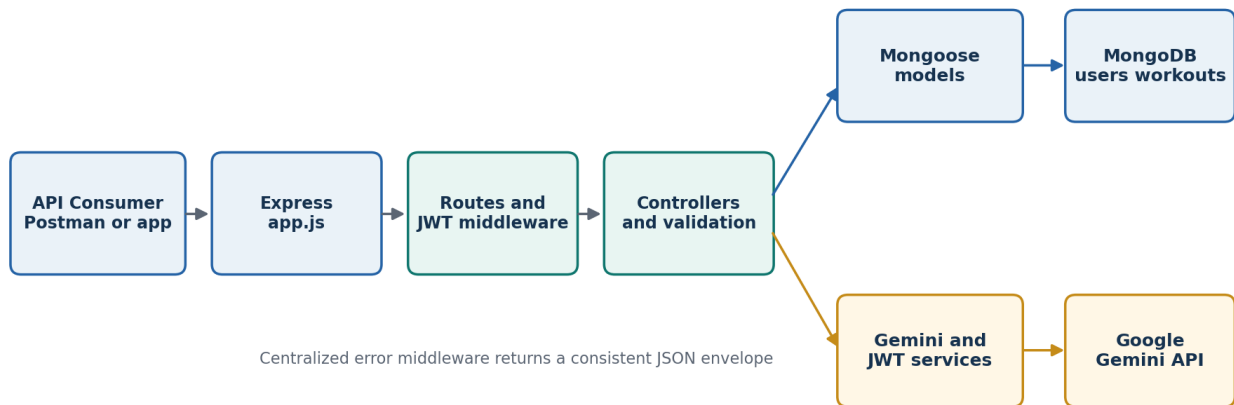


Figure 3 Implemented layered backend architecture

The Express gateway receives the request and applies common middleware. Feature routes select a controller and enforce JWT protection where needed. Controllers call validation utilities, Mongoose models and services. MongoDB stores users and workouts; Google Gemini provides structured AI output. The centralized error middleware converts failures into a consistent JSON envelope.

3.1 Request sequence

Protected request sequence

Example for a workout create update search or delete request

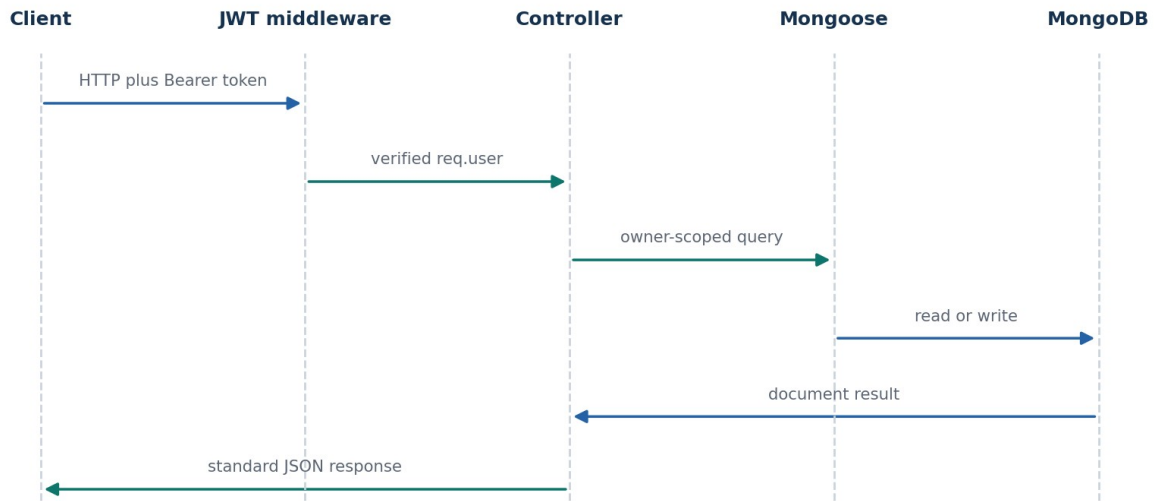


Figure 4 Protected owner-scoped request sequence

3.2 Component responsibilities

Component	Responsibility
server.js	Validate JWT configuration, connect MongoDB, start HTTP listener and shut down safely.
app.js	Configure CORS and parsers, expose health, mount routes and register error handlers.
auth module	Register, login, profile and public email availability.
workout module	Owner-scoped CRUD, history ordering and search.
AI module	Recommendation, insights, rate limit and provider failure mapping.
models	Schema validation, indexes, password hook and safe JSON output.
services	Centralized JWT behavior and structured Gemini generation.
tests and Postman	Automated and guided verification assets.

4 Database Design

MongoDB document relationship

Mongoose schemas and implemented validation boundaries

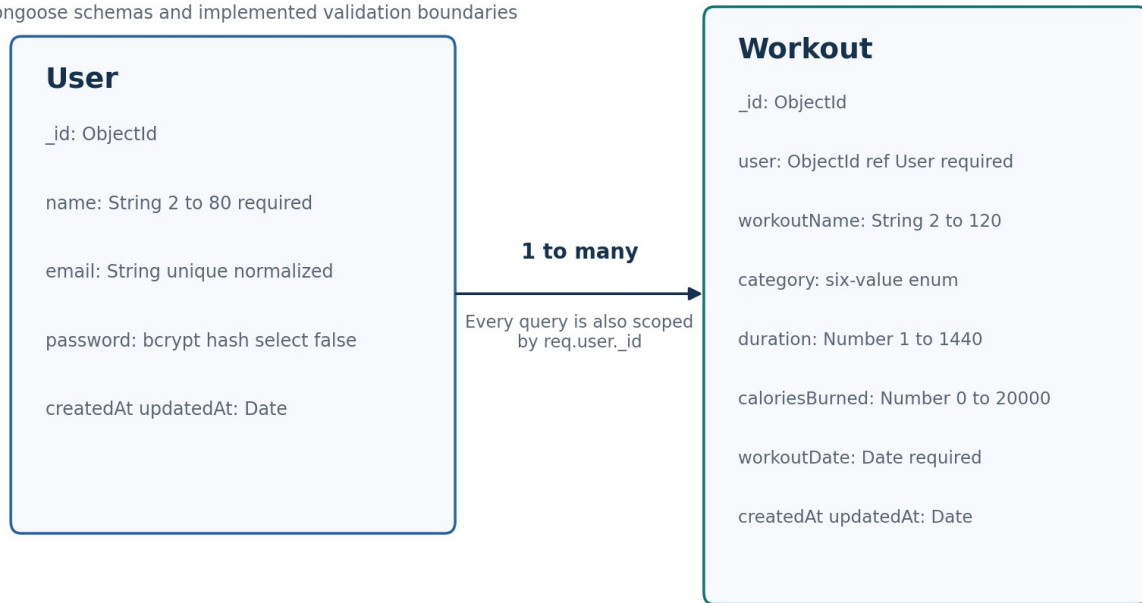


Figure 5 MongoDB collections and relationship

A user owns zero or more workouts. The relationship is stored in Workout.user as a required ObjectId reference. Queries always add the authenticated user's identifier, which avoids relying on a user id in the request body.

Collection	Important constraints	Indexes
users	Validated unique normalized email; bcrypt password; timestamps; password excluded from standard selection	Unique email index
workouts	Required owner, name, category, duration, calories and date; timestamps	user plus workoutDate plus createdAt; user plus category

5 API Specification

Method	Endpoint	Access	Purpose	Main statuses
GET	/api/health	Public	Service health and runtime metadata	200
POST	/api/auth/register	Public	Create user and issue JWT	201, 400, 409
POST	/api/auth/login	Public	Verify credentials and issue JWT	200, 400, 401
GET	/api/auth/email-available	Public	Check normalized email availability	200, 400
GET	/api/auth/profile	JWT	Return authenticated user profile	200, 401
POST	/api/workouts	JWT	Create an owner-scoped workout	201, 400, 401
GET	/api/workouts	JWT	List history and optional filters	200, 400, 401
GET	/api/workouts/search	JWT	Search by name, category or UTC date	200, 400, 401
GET	/api/workouts/:id	JWT	Read one owned workout	200, 400, 404
PUT	/api/workouts/:id	JWT	Update allowed workout fields	200, 400, 404
DELETE	/api/workouts/:id	JWT	Delete one owned workout	200, 400, 404
POST	/api/ai/workout-recommendation	JWT	Generate structured workout plan	200, 400, 429, 5xx
POST	/api/ai/fitness-insights	JWT	Generate submitted or database-derived insights	200, 400, 429, 5xx

5.1 Request examples

```
POST /api/auth/register
{
  "name": "Demo User",
  "email": "demo@example.com",
  "password": "<private value>"
}
```

```

POST /api/workouts
Authorization: Bearer <token>
{
  "workoutName": "Morning Run",
  "category": "Running",
  "duration": 30,
  "caloriesBurned": 280,
  "workoutDate": "2026-09-20"
}

POST /api/ai/workout-recommendation
Authorization: Bearer <token>
{
  "age": 22,
  "fitnessGoal": "Weight Loss",
  "experienceLevel": "Beginner"
}

POST /api/ai/fitness-insights
Authorization: Bearer <token>
{ "useDatabaseStats": true }

```

5.2 Response design

Successful responses include success, message and data. Errors include success false, a human-readable message and an errors array. AI operational failures may additionally include a safe code, attempt count and retryAfterSeconds.

6 Implementation

Implemented server module map

File groups present in the uploaded backend ZIP

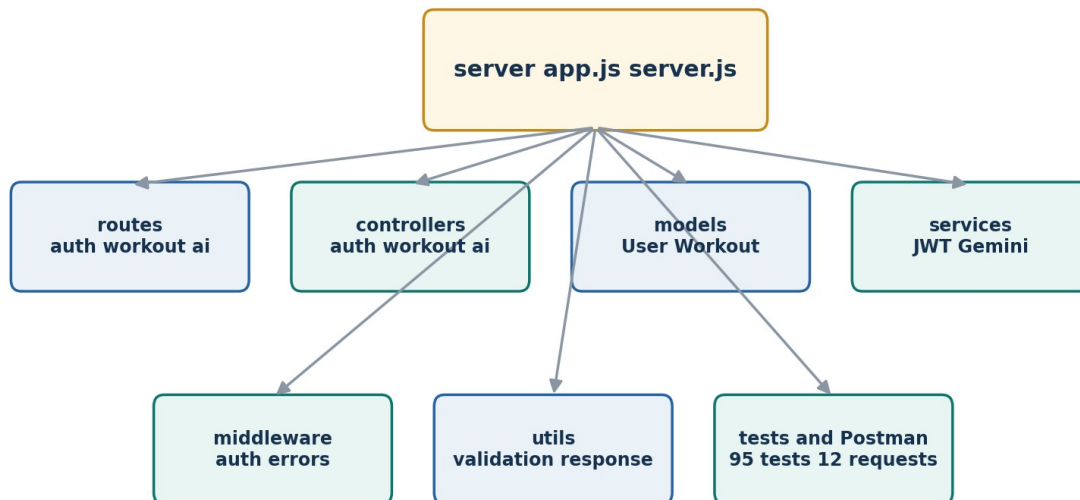


Figure 6 Implemented source modules

The project is packaged as a private Node.js module named ai-fittrack-api. The server uses Express 5 and Mongoose 9 and declares Node.js 20 or later. package-lock.json supports reproducible installation. No private .env file is included.

6.1 Authentication

User passwords are hashed before storage. Registration and login return safe user fields and a signed JWT. Protected middleware accepts only Bearer tokens, verifies the issuer and signature, checks the identifier and reloads the user. A deleted or unknown user therefore cannot keep using an old token.

6.2 Workout management

Workout validation is performed before model operations, and the model provides a second validation boundary. Listing is ordered by workout date and creation time. Search supports an escaped partial name, canonical category, valid UTC calendar date and a result limit clamped to 1-500.

6.3 Gemini AI

Gemini prompts explicitly restrict the model to general fitness guidance and instruct it not to assume medical conditions or create extreme programs. Response schemas make the output predictable for clients. The service handles timeouts, quota pressure, unavailable services, bad keys, model errors and malformed output.

7 Security and Reliability

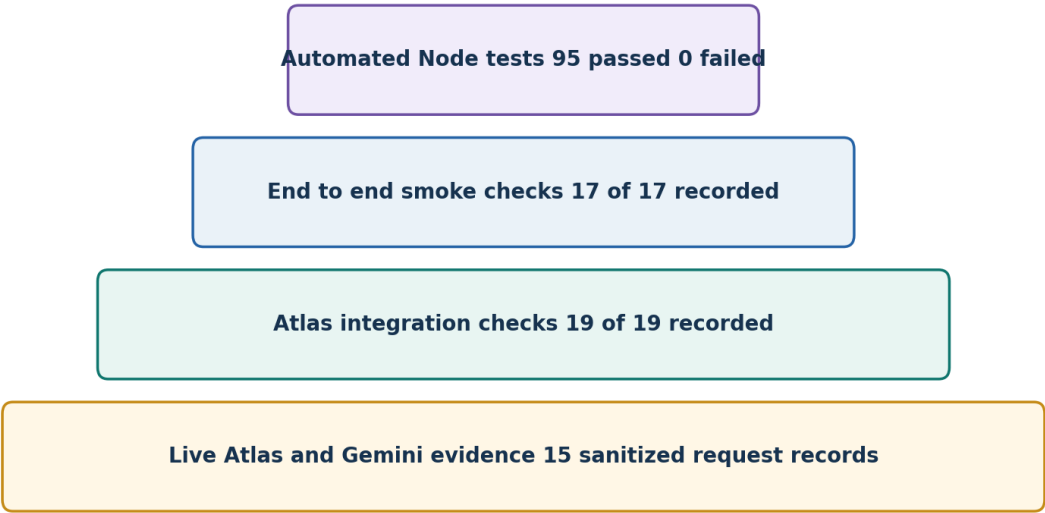
Control	Implementation
Password storage	bcrypt with cost 12; password excluded from normal model selection and serialized output.
Token validation	JWT secret required, issuer checked, expiry supported and active user reloaded.
Data isolation	Every workout query is owner-scoped by req.user_id.
Input safety	Allow-lists, numeric boundaries, ObjectId checks, regex escaping and malformed JSON handling.
CORS	Origins are derived from CLIENT_URL; unauthorized browser origins receive 403.
AI resilience	Timeout, bounded retry, Retry-After support, safe error mapping and per-user request limit.

Control	Implementation
Secret handling	Only .env.example is included; private values are loaded locally.
Operational behavior	Database connection must succeed before listen; SIGINT and SIGTERM close services.

8 Testing and Results

Verification layers and recorded results

Counts are taken from supplied evidence; no new result is invented



Coverage includes auth validation security isolation CRUD search AI retries and error handling

Figure 7 Recorded verification layers

Test file	Cases	Coverage
ai.test.js	23	AI validation, structured responses, database stats, failures, rate limiting and regression
auth.test.js	23	Registration, login, profile, token failures, duplicate and password behavior
geminiRetry.test.js	13	Transient retries, Retry-After, bounded backoff, non-retryable failures and parsing

Test file	Cases	Coverage
security.test.js	8	CORS, ownership, secret handling, password exposure and security boundaries
validation.test.js	16	Authentication, workout and AI input normalization and validation
workouts.test.js	12	CRUD, search, ordering, isolation, validation and UTC date behavior
Total	95	Recorded automated run: 95 passed and 0 failed

8.1 Live result highlights

- Registration returned 201 with a token and safe user fields; the password hash did not appear.
- Login and authorized profile returned 200; profile without a token returned 401.
- Workout create, list, retrieve, update, name search, category search, date search and delete succeeded; follow-up retrieval after delete returned 404.
- The recommendation endpoint returned a structured seven-day Gemini result for a beginner weight-loss profile.
- Database insights computed 3 workouts, a 38.3-minute average and 780 calories exactly from three temporary owner records.
- Temporary evidence users and workouts were removed after the verification flows according to the supplied cleanup records.

8.2 Evidence qualification

The live evidence is sanitized text, not a collection of screenshots. It also contains two response-shape differences from the uploaded source: the recorded delete payload is null whereas the current controller returns the deleted workout, and the submitted-statistics source label is recorded as manual whereas the current controller uses submitted. These do not invalidate the verified features, but the exact current responses should be re-captured from the final GitHub commit.

9 Outcomes Limitations and Future Scope

9.1 Outcomes

The project delivers the required backend capabilities with clear separation of concerns, secure owner isolation, structured AI output and a substantial verification suite. The response contracts are suitable for a future browser or mobile client.

9.2 Current limitations

- No user interface is included in this backend-only submission.
- The in-process AI rate-limit map is not shared across multiple server instances.
- There is no refresh-token or token-revocation store.
- Workout lists use a bounded limit but do not implement cursor pagination.
- Gemini guidance is general fitness information and is not medical advice.
- A public deployment URL and final video URL have not been supplied.

9.3 Future enhancements

- Frontend dashboard with accessible charts and workout forms.
- Pagination, sorting options and aggregated progress endpoints.
- Distributed rate limiting and structured operational telemetry.
- Refresh-token rotation, account recovery and configurable role support.
- Nutrition, BMI, reminders and wearable integration after separate privacy review.

10 Conclusion

AI FitTrack meets the official backend objectives with secure authentication, MongoDB workout management, search and Gemini-assisted guidance. The implementation and supplied evidence support the functional completion claim. Final submission readiness depends on adding official student metadata, publishing the repository, capturing synchronized final evidence and recording the demonstration video.

Appendix A Source and evidence register

Artifact	Use in this report
AI-FitTrack-backend(1).zip	Authoritative uploaded source baseline.
AI FitTrack(5).docx	Official project description, architecture, schemas, features and execution requirements.
AI-FitTrack-submission-evidence.zip	Sanitized live auth, workout and Gemini records plus requirements and README verification.
server/tests/*.test.js	95 present automated test definitions across six files.
AI-FitTrack.postman_collection.json	12 ordered manual requests.

Appendix B Safe local configuration checklist

- Create server/.env locally from server/.env.example.
- Use a long random JWT_SECRET and a restricted MongoDB account.

- Restrict Atlas network access rather than leaving an unrestricted rule permanently.
- Keep GEMINI_API_KEY server-side and rotate it if it is ever exposed.
- Verify git status and repository history contain no private environment file.

Items requiring student confirmation

Pending item	What to provide
Student identity	Name, NM ID, register or roll number and department were not present in the supplied project files.
Institution details	College name, academic year, guide or mentor name and official submission date need confirmation.
Team details	Team name, member list and responsibility allocation were not supplied; no ownership is assigned in these documents.
Repository	Insert the final public GitHub repository URL after upload.
Deployment	Insert the deployed API base URL if the evaluator requires a hosted demonstration.
Demonstration video	Record the video using Phase 8 and insert the final shareable video URL.